

Computer Architecture

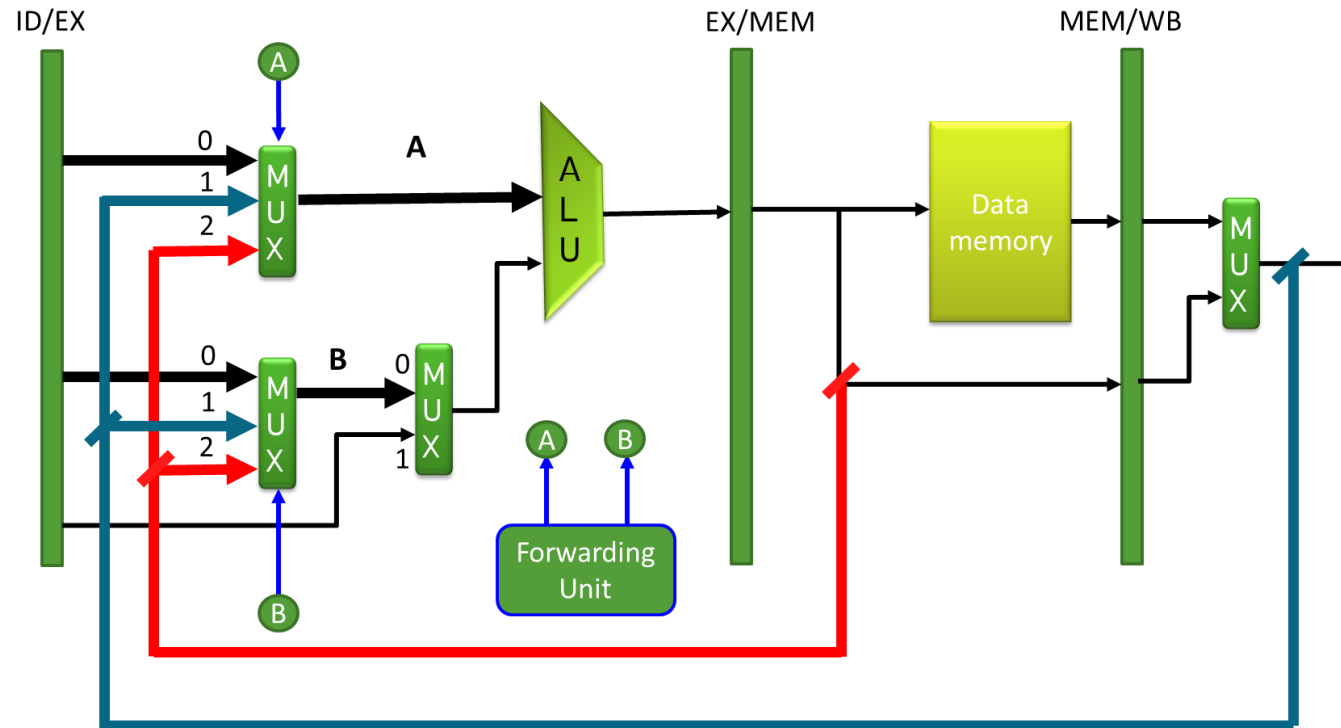
CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 21

Forwarding Unit



1. ADD x5, x6, x7
2. SUB x8, x5, x9
3. AND x10, x5, x11

Where to find the ALU result?

✓ No forwarding: If there is no hazard, the ALU's operands will come from the register file, just like before.

EX-Hazard or EX/MEM-ALU forwarding: Result will come from EX/MEM pipeline

MEM-Hazard or MEM/WB-ALU forwarding: Result will come from MEM/WB pipeline

Detecting Data Hazards

MEM Hazard: MEM/WB-ALU forwarding

- A data hazard may occur between an instruction in the EX stage and the instruction from **two cycles ago** if:
 - The destination is one of the ALU source registers in the EX stage, and
 - The previous to previous instruction will write to the register file, and
 - The destination register is not 'x0'

For detecting and handling MEM hazard for the first ALU source.

- if ((MEM/WB.Register_rd == ID/EX.Register_rs1) && MEM/WB.RegWrite && MEM/WB.Register_rd != 0)

 then ForwardA = 01

The second ALU source is treated in a similar way

- if ((MEM/WB.Register_rd == ID/EX.Register_rs2) && MEM/WB.RegWrite && MEM/WB.Register_rd != 0)

 then ForwardB = 01

	rd	rs1	rs2
1) ADD	x6	x7	x8
2) SUB	x9	x7	x5
3) AND	x4	x6	x10

ForwardA = 01

1) BNE	x6	x7	dest
2) -----			
3) SUB	x9	x8	x5

ForwardA/ForwardB = 0

1) SW	x7	100(x8)
2) -----		
3) SUB	x9	x5, x6

ForwardA/ForwardB = 0

Detecting Data Hazards

MEM Hazard: MEM/WB-ALU forwarding

- A data hazard may occur between an instruction in the EX stage and the instruction from two cycles ago if:
 - The destination is one of the ALU source registers in the EX stage, and
 - The previous to previous instruction will write to the register file, and
 - The destination register is not R0

For detecting and handling MEM hazard for the first ALU source.

- if $((\text{MEM/WB.Register_rd} == \text{ID/EX.Register_rs1}) \ \&\& \ \text{MEM/WB.RegWrite} \ \&\& \ \text{MEM/WB.Register_rd} \neq 0)$

$\&\&$

$! ((\text{EX/MEM.Register_rd} == \text{ID/EX.Register_rs1}) \ \&\& \ \text{EX/MEM.RegWrite} \ \&\& \ \text{EX/MEM.Register_rd} \neq 0)$
 then ForwardA = 01

For second ALU source

- if $((\text{MEM/WB.Register_rd} == \text{ID/EX.Register_rs2}) \ \&\& \ \text{MEM/WB.RegWrite} \ \&\& \ \text{MEM/WB.Register_rd} \neq 0)$

$\&\&$

$! ((\text{EX/MEM.Register_rd} == \text{ID/EX.Register_rs2}) \ \&\& \ \text{EX/MEM.RegWrite} \ \&\& \ \text{EX/MEM.Register_rd} \neq 0)$
 then ForwardB = 01

	rd	rs1	rs2
1) ADD	x5	x6	x7
2) SUB	x5	x8	x5
3) AND	x9	x5	x10

ForwardA = 01

ForwardA = 10



1) ADD	x5	x5	x5
2) ADD	x5	x5	x5
3) ADD	x5	x5	x5

ForwardA = ?

ForwardB = ?

Summary - Conditions

EX Hazard: EX/MEM-ALU forwarding

- 1a) if ((EX/MEM.Register_rd == ID/EX.Register_rs1) && EX/MEM.RegWrite && EX/MEM.Register_rd != 0)
then ForwardA = 10
- 1b) if ((EX/MEM.Register_rd == ID/EX.Register_rs2) && EX/MEM.RegWrite && EX/MEM.Register_rd != 0)
then ForwardB = 10

MEM Hazard: MEM/WB-ALU forwarding

- 2a) if (((MEM/WB.Register_rd == ID/EX.Register_rs1) && MEM/WB.RegWrite && MEM/WB.Register_rd != 0)
&& ! ((EX/MEM.Register_rd == ID/EX.Register_rs1) && EX/MEM.RegWrite && EX/MEM.Register_rd != 0))
then ForwardA = 01
- 2b) if (((MEM/WB.Register_rd == ID/EX.Register_rs2) && MEM/WB.RegWrite && MEM/WB.Register_rd != 0)
&& ! ((EX/MEM.Register_rd == ID/EX.Register_rs2) && EX/MEM.RegWrite && EX/MEM.Register_rd != 0))
then ForwardB = 01

Forwarding Unit Design

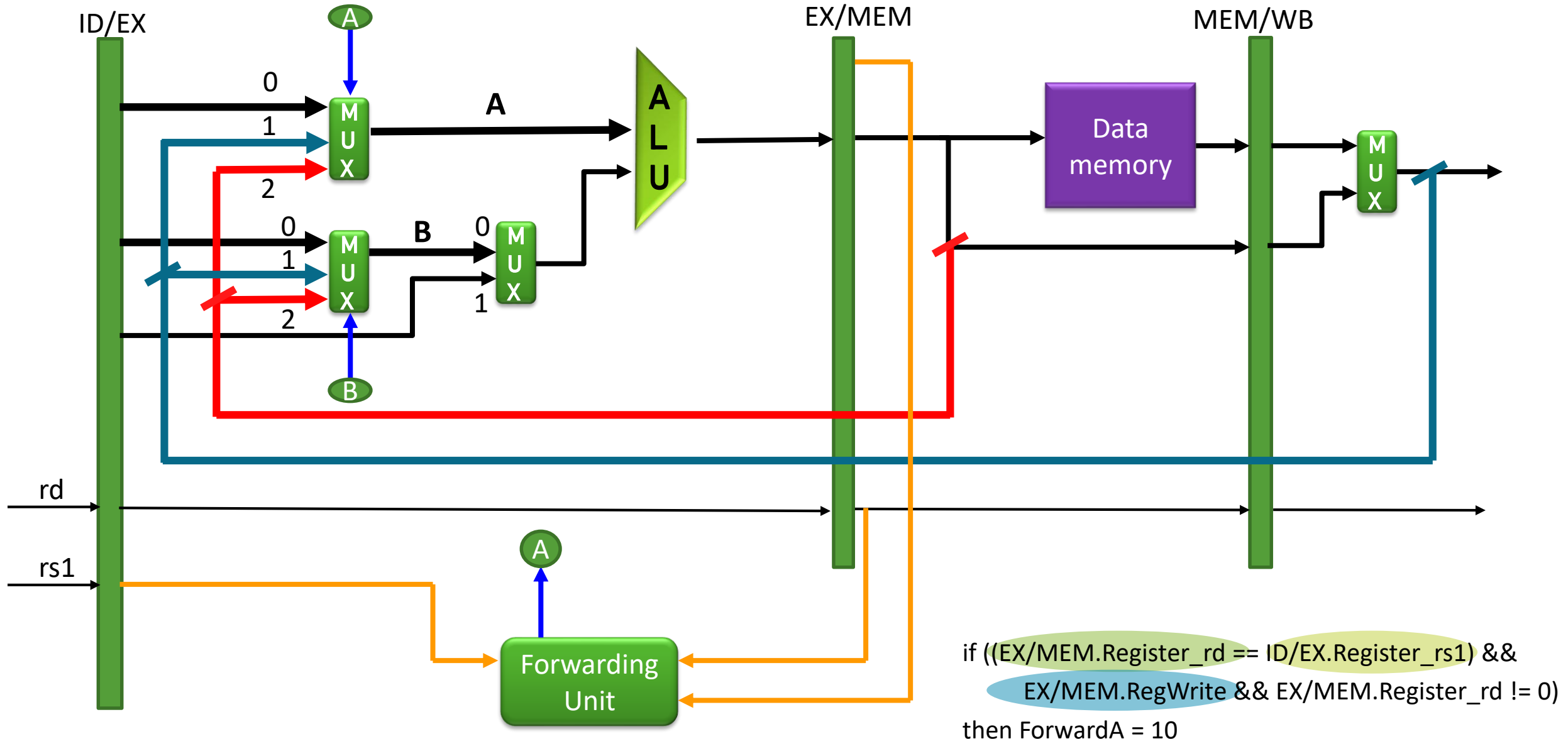
Forward – A

```
if ((EX/MEM.Register_rd == ID/EX.Register_rs1) && EX/MEM.RegWrite && EX/MEM.Register_rd != 0)
    ForwardA = 10
else if ((MEM/WB.Register_rd == ID/EX.Register_rs1) && MEM/WB.RegWrite && MEM/WB.Register_rd != 0)
    ForwardA = 01
else
    ForwardA=0
```

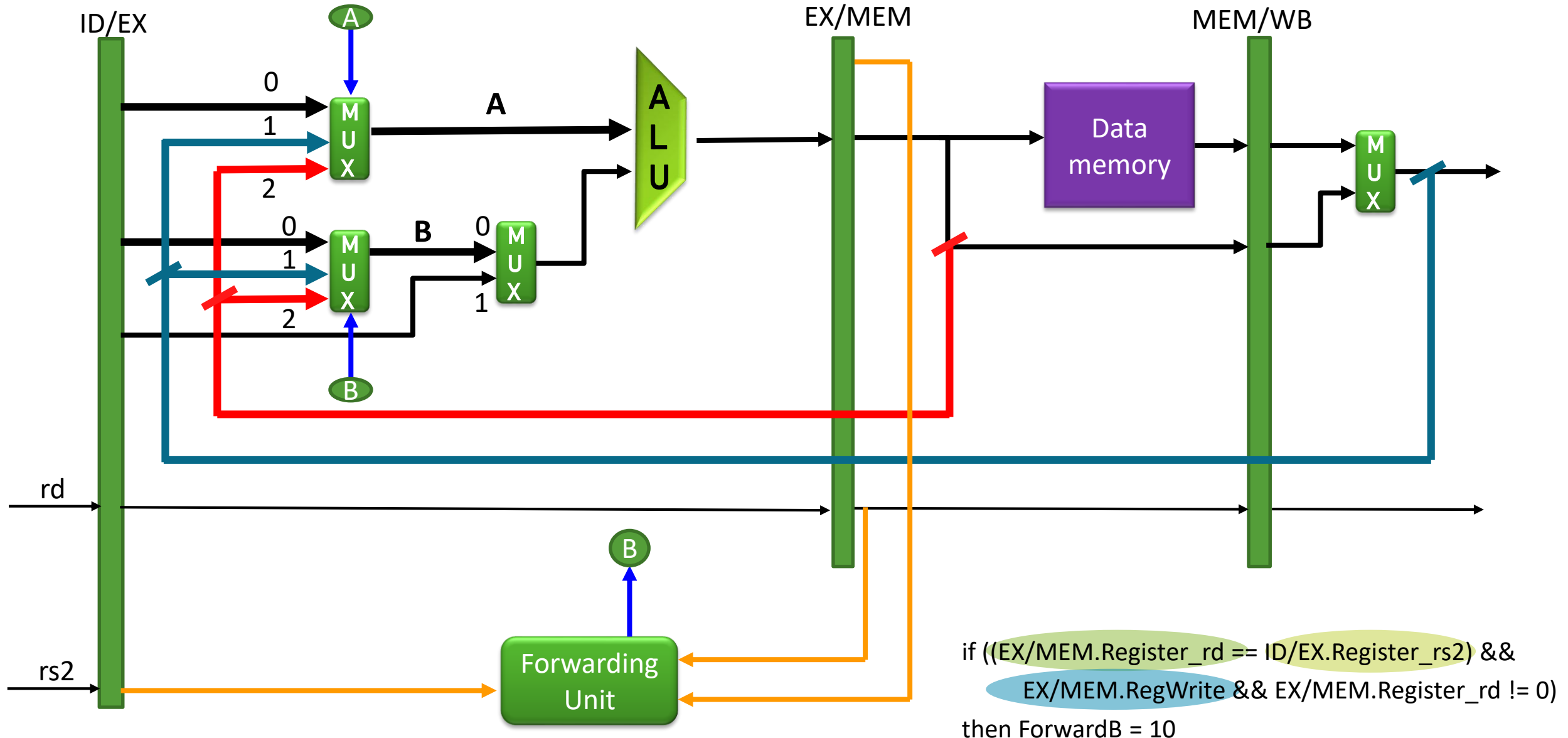
Forward – B

```
if ((EX/MEM.Register_rd == ID/EX.Register_rs2) && EX/MEM.RegWrite && EX/MEM.Register_rd != 0)
    ForwardB = 10
else if ((MEM/WB.Register_rd == ID/EX.Register_rs2) && MEM/WB.RegWrite && MEM/WB.Register_rd != 0)
    ForwardB = 01
else
    ForwardB = 0
```

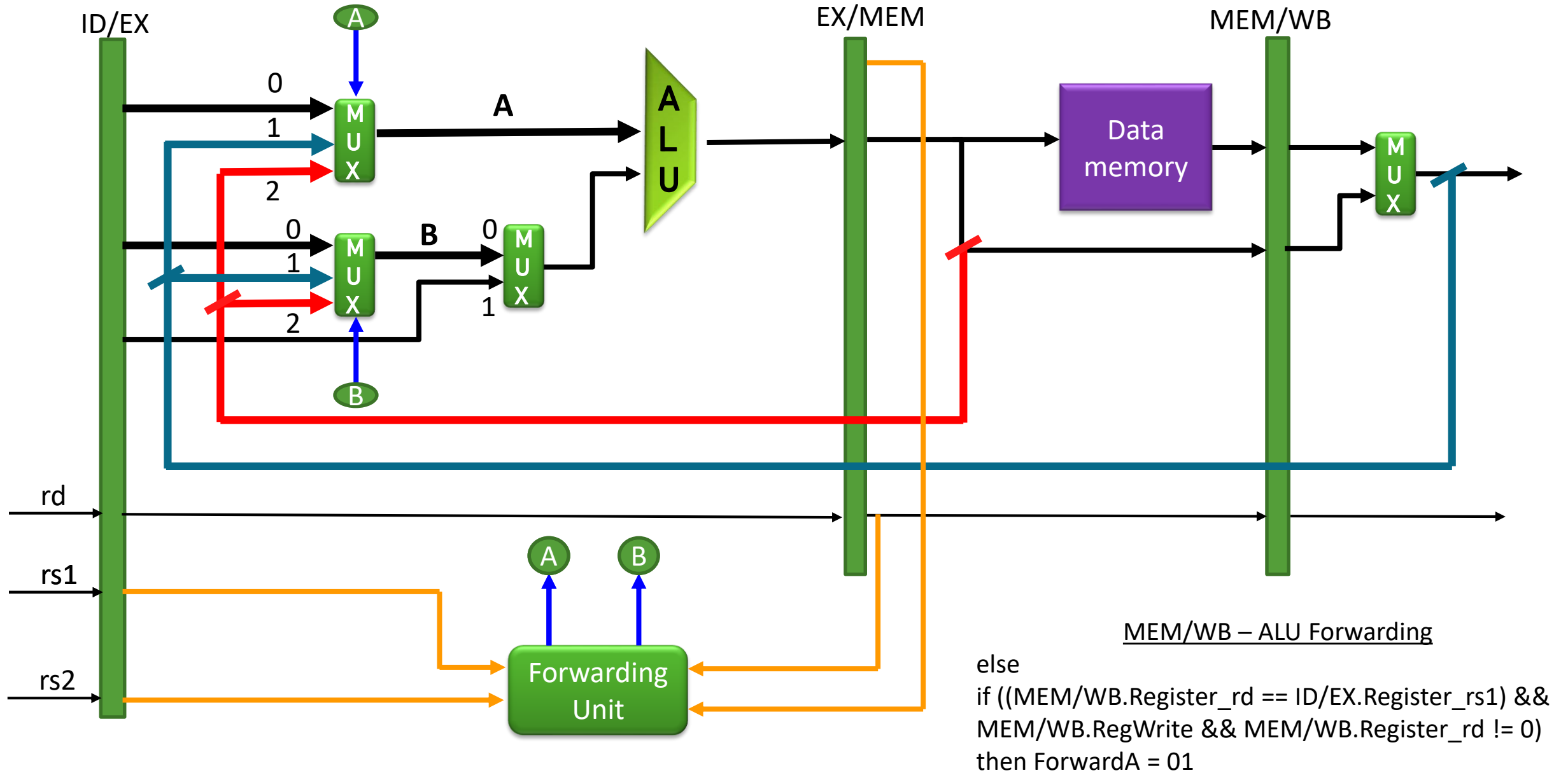
Designing Forwarding Unit

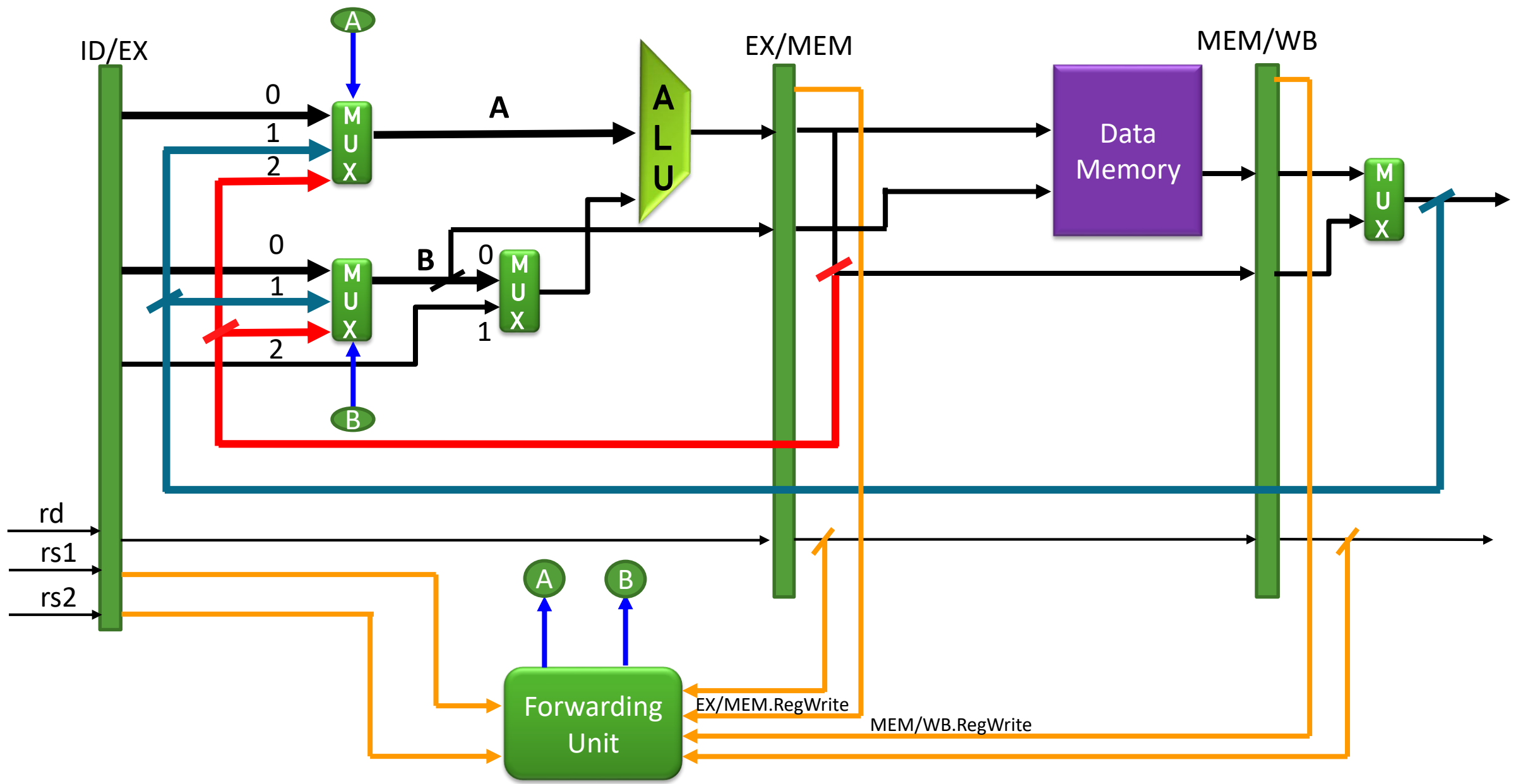


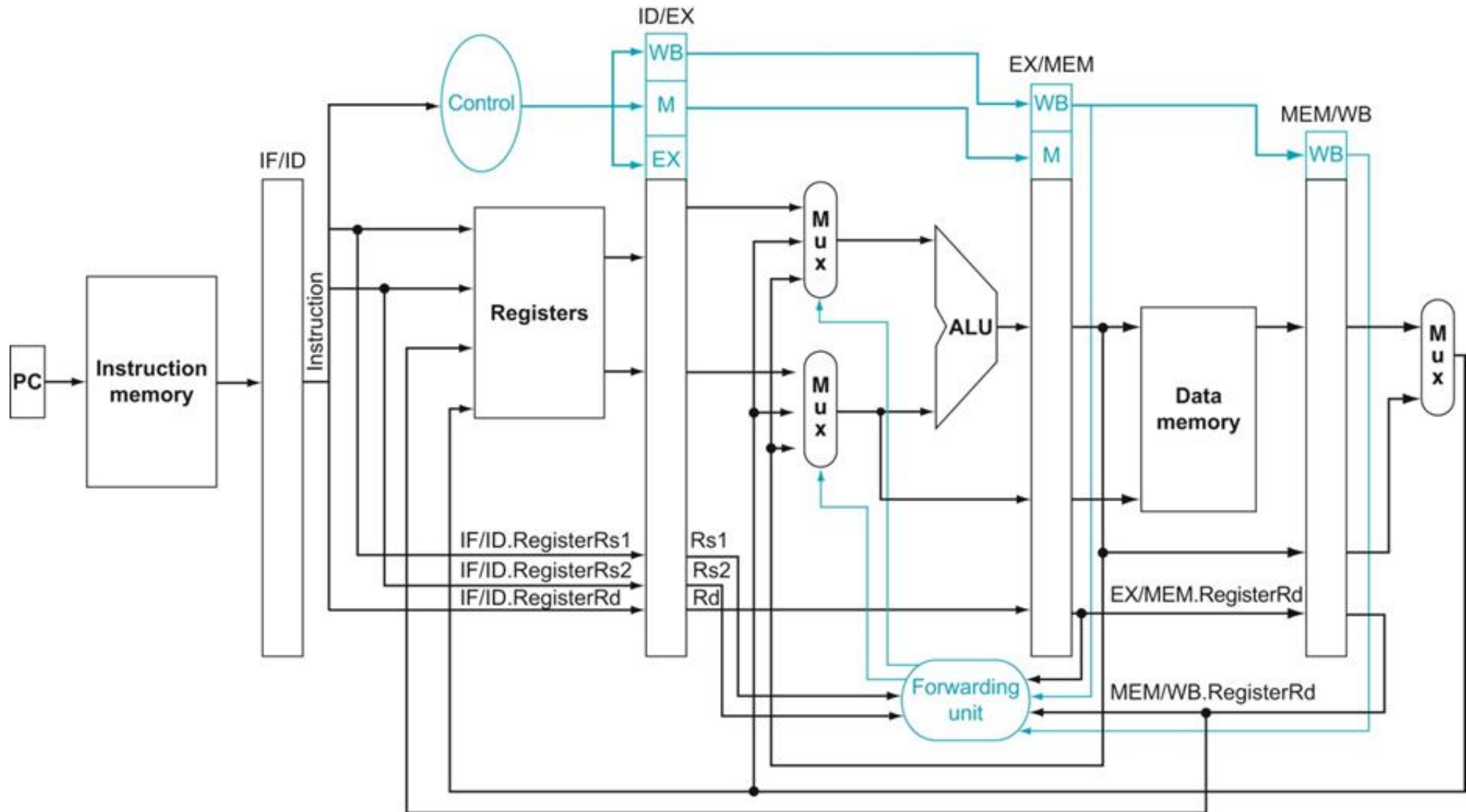
Designing Forwarding Unit



Designing Forwarding Unit



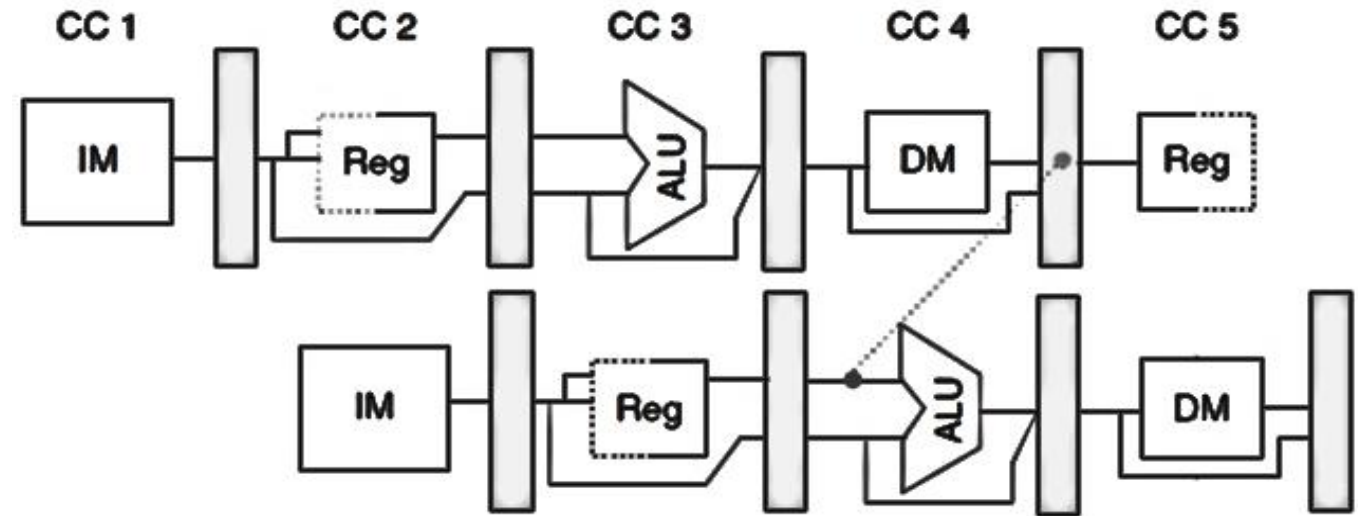




Load-Use Hazards

LD x5, 0(x7)

SUB x8, x5, x9

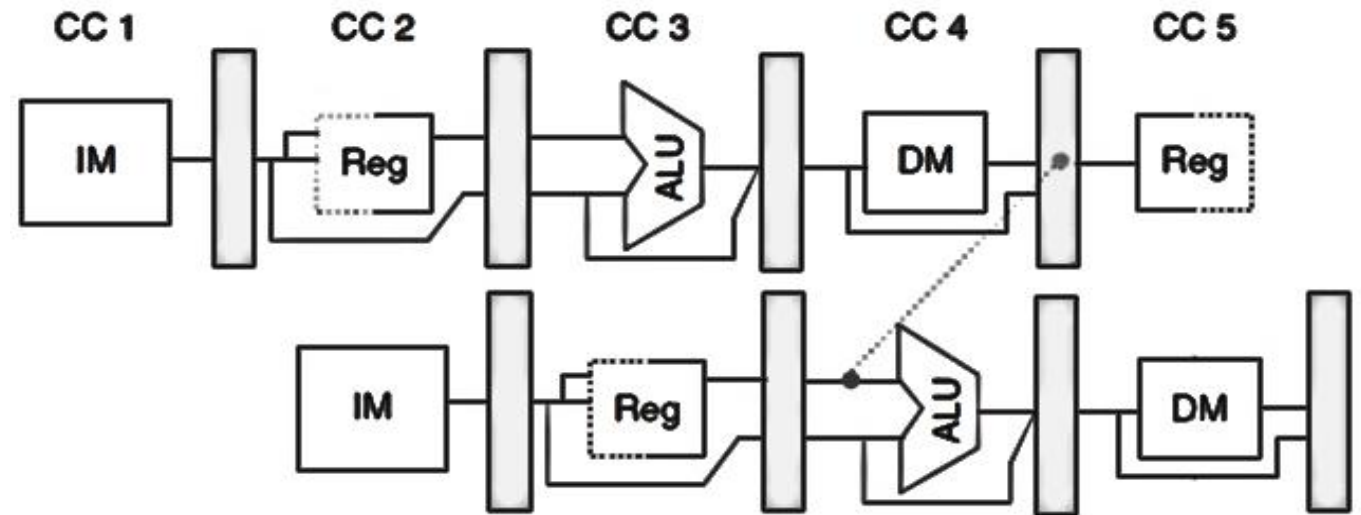


- A load use hazard occurs between the current instruction and the preceding instruction if:
 - The preceding instruction is a load instruction and
 - The load's destination register is one of the current instructions source registers. (used in EX-stage)

Load-Use Hazards

LD x5, 0(x7)

SUB x8, x5, x9



- After insertion of 1-cycle stall, the forwarding logic can handle the dependence and execution proceeds.
- If there are no forwarding paths, then the instruction sequence would need another stall cycle.
- If the current instruction in the ID stage is stalled, then the next instruction in the IF stage must also be stalled; otherwise, we would lose the fetched instruction.

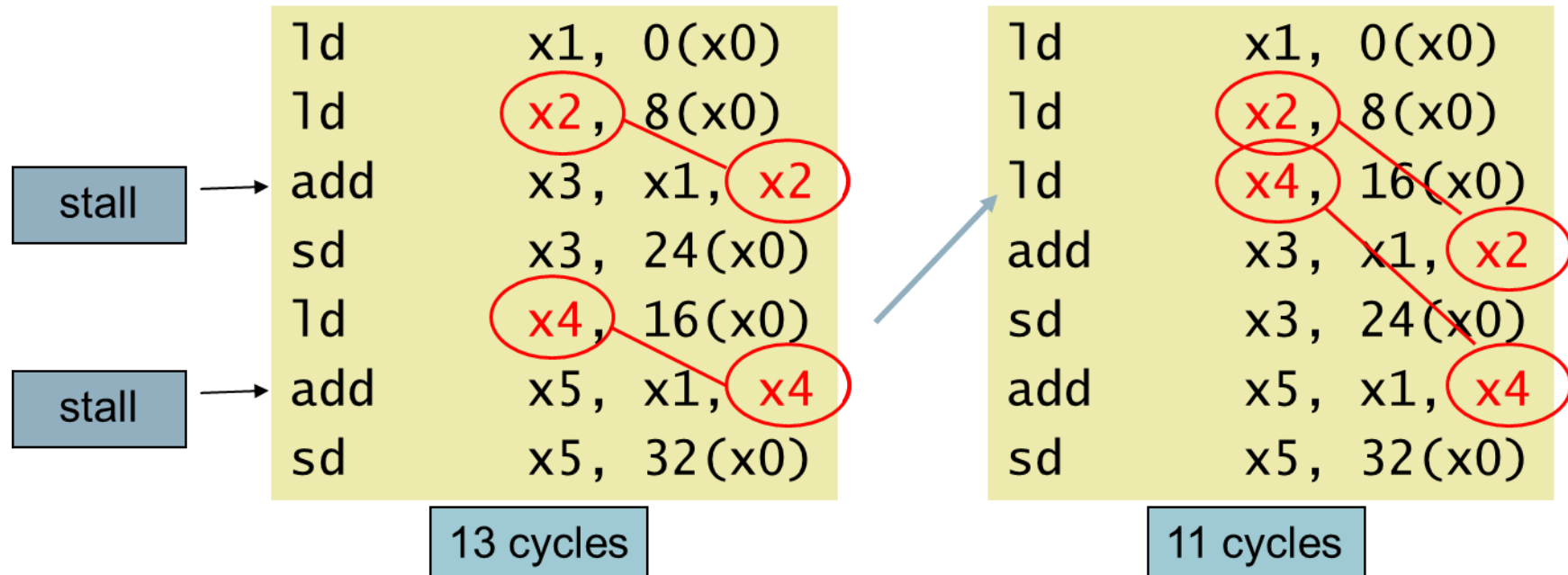
How to Insert Stall Cycles?

- Software solution (Compile time)

- Compiler checks for sequences where load is followed by any instruction using its result (in EX stage) and 'NOP' (no operation) instructions are placed in between these two instructions.

- OR the compiler could reorder the code

- e.g. $e = a + b;$
 $f = a + c;$



How to Insert Stall Cycles?

- Hardware solution (Runtime)

- A special hardware unit is required to detect such sequences at runtime. This unit is called **hazard detection unit** or pipeline interlock.
- The unit is responsible to detect the pipeline hazard and insert one cycle or more stalls, just like what a 'nop' instruction or multiple 'nop' instructions would have done.
- Preventing an instruction from making progress is accomplished simply by nullifying its control signals and repeating the fetch and decode process.

How early can a load-use hazard be detected?

	1	2	3	4	5	6	7	8	9
LD x5, 0(x7)	IF	ID	EX	MEM	WB				
SUB x8, x5, x9		IF	ID	EX	MEM	WB			
AND x10, x5, x11			IF	ID	EX	MEM	WB		
OR x12, x5, x13				IF	ID	EX	MEM	WB	

The hazard detection unit operates during the ID stage so that it can insert the stall between the load and its use.

	1	2	3	4	5	6	7	8	9
LD x5, 0(x7)	IF	ID	EX	MEM	WB				
SUB x8, x5, x9		IF	ID	ID(again)	EX	MEM	WB		
AND x10, x5, x11			IF	IF(again)	ID	EX	MEM	WB	
OR x12, x5, x13					IF	ID	EX	MEM	WB

How to detect a load-use hazard?

			1	2	3	4	5	6	7	8	9
LD	x5,	0(x7)	IF	ID	EX	MEM	WB				
SUB	x8,	x5, x9		IF	ID	ID(again)	EX	MEM	WB		
AND	x10,	x5, x11			IF	IF(again)	ID	EX	MEM	WB	
OR	x12,	x5, x13					IF	ID	EX	MEM	WB

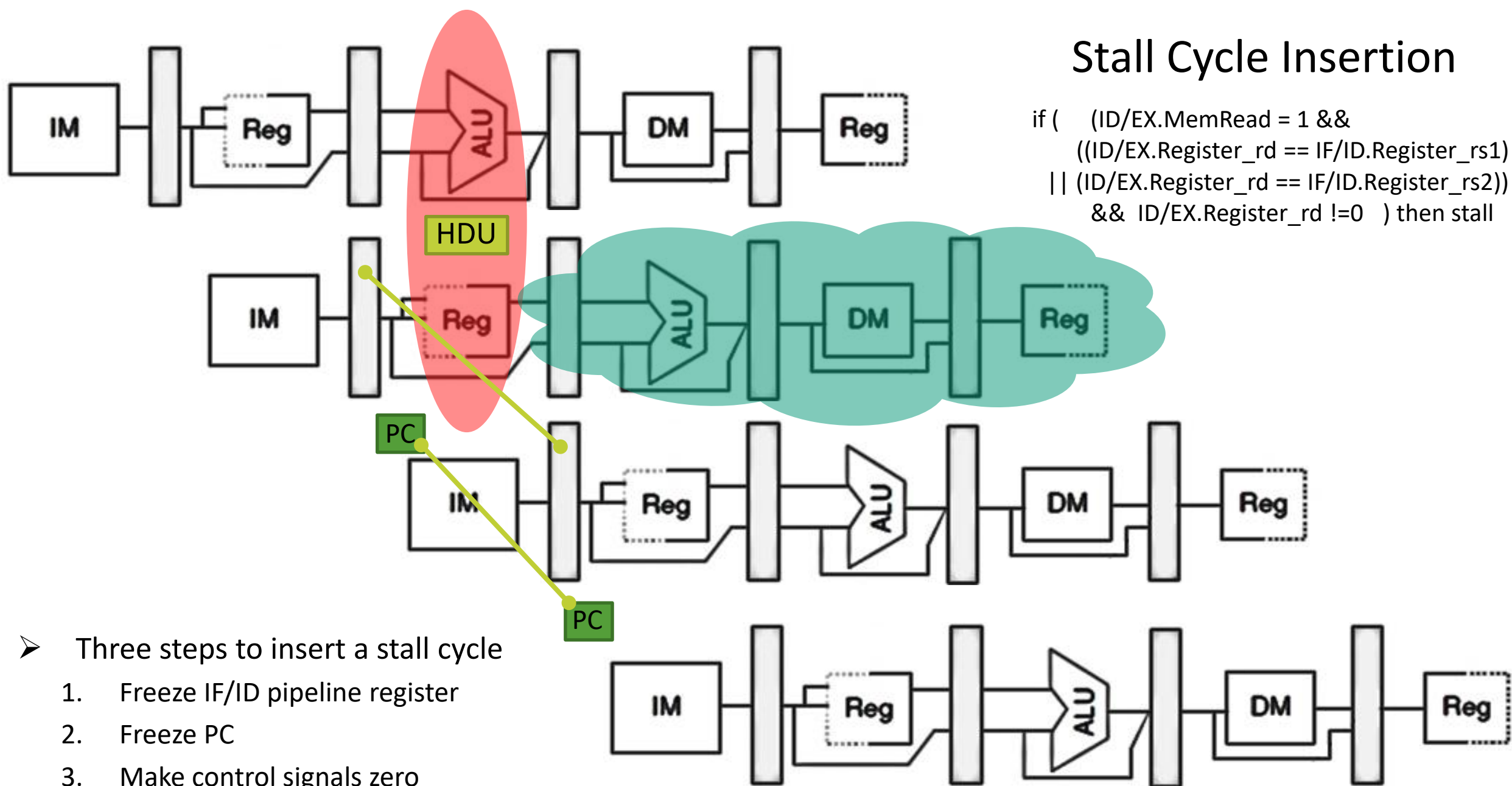
The set of equations to test the load-use hazard condition:

```

if (      ID/EX.MemRead = 1  &&
        ( (ID/EX.Register_rd == IF/ID.Register_rs1)  ||
          (ID/EX.Register_rd == IF/ID.Register_rs2) )
        && ID/EX.Register_rd !=0
    )
    then stall
    
```

Stall Cycle Insertion

```
if ( (ID/EX.MemRead = 1 &&  
      ((ID/EX.Register_rd == IF/ID.Register_rs1)  
      || (ID/EX.Register_rd == IF/ID.Register_rs2))  
      && ID/EX.Register_rd != 0 ) then stall
```



➤ Three steps to insert a stall cycle

1. Freeze IF/ID pipeline register
2. Freeze PC
3. Make control signals zero

Hazard Detection Unit – what is a stall cycle?

- Three sub-operations that are needed for inserting stall cycle
 1. Freeze IF/ID pipeline register
 - This enables the last fetched instruction (currently in the decode stage) to be re-decoded and its registers re-read. It also prevents the recently fetched instruction from being saved.
 2. Freeze PC
 - This enables the PC to point to the same instruction which was fetched earlier so that it is fetched again.
 3. Make all Control Signals Zero
 - Making all control signals zero means that no memory or register location would be updated. Moreover nothing would be done for this fetched and decoded instruction in the later stages i.e. EX, MEM and WB. Making it same as a 'nop' instruction.

Computer Architecture

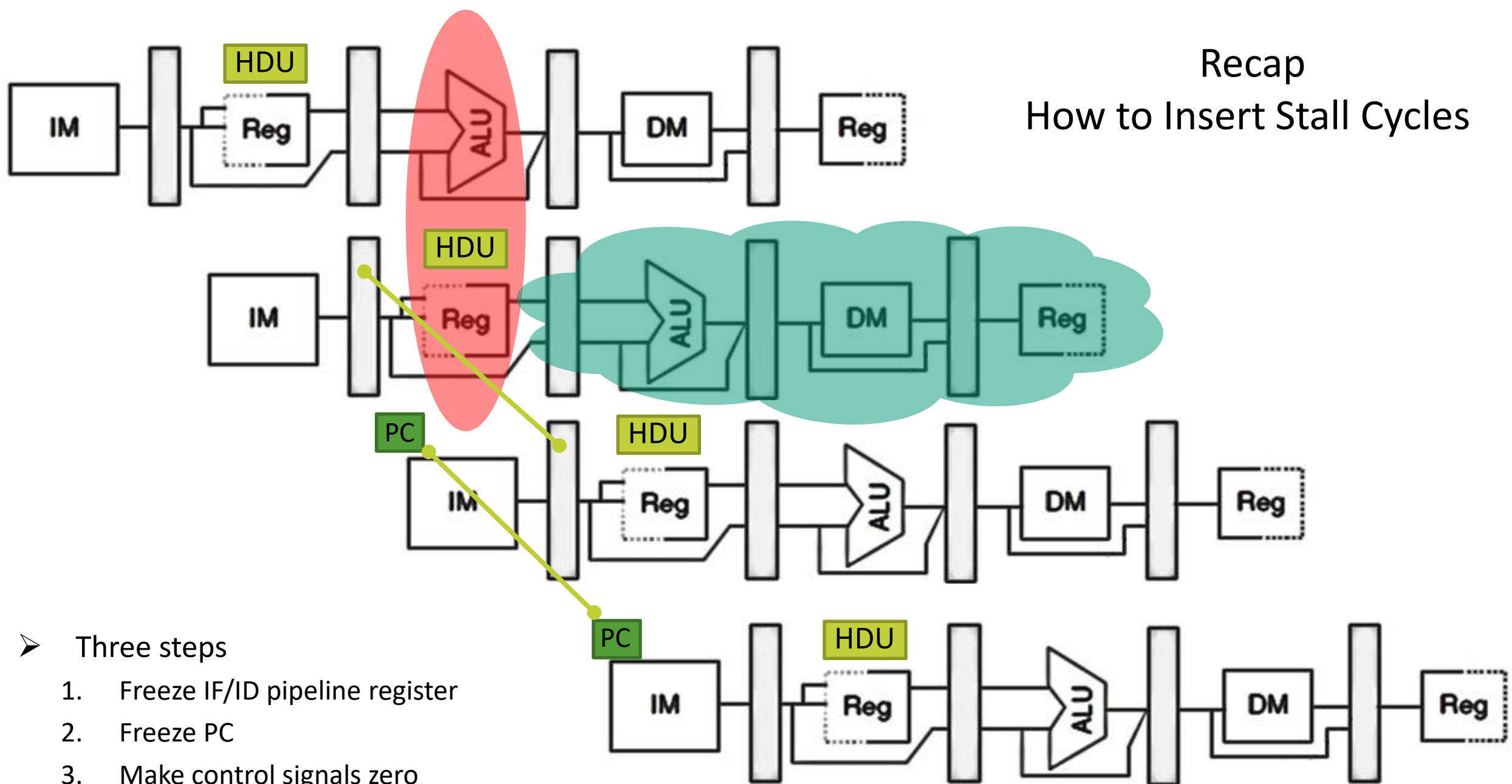
CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 22

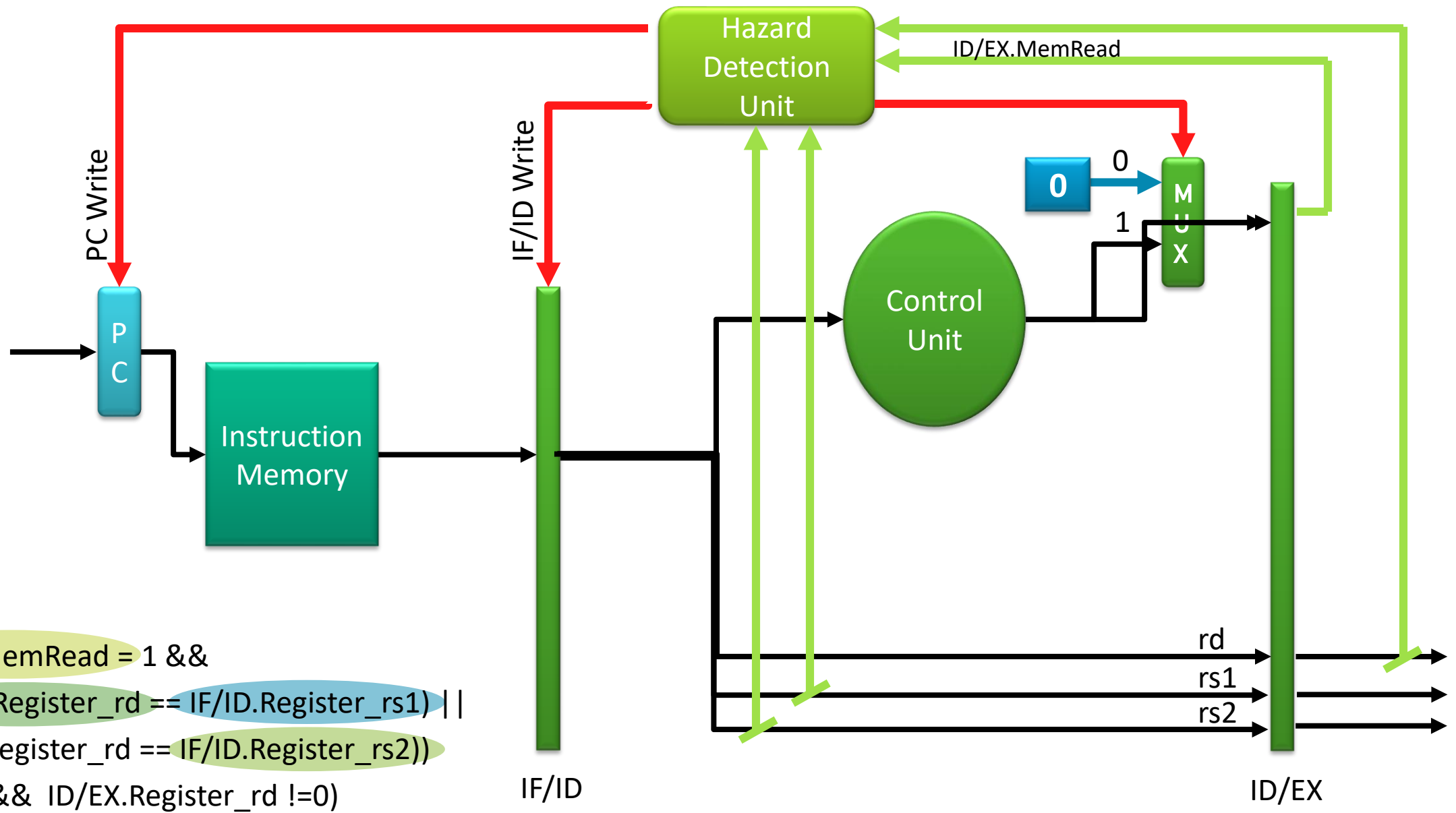
Recap How to Insert Stall Cycles



Recap - How to detect a load-use hazard?

The set of equations to test the load-use hazard condition:

```
if (      ID/EX.MemRead = 1  &&  
        ( (ID/EX.Register_rd == IF/ID.Register_rs1)  ||  
          (ID/EX.Register_rd == IF/ID.Register_rs2) )  
        && ID/EX.Register_rd != 0  
    )  
    then stall
```



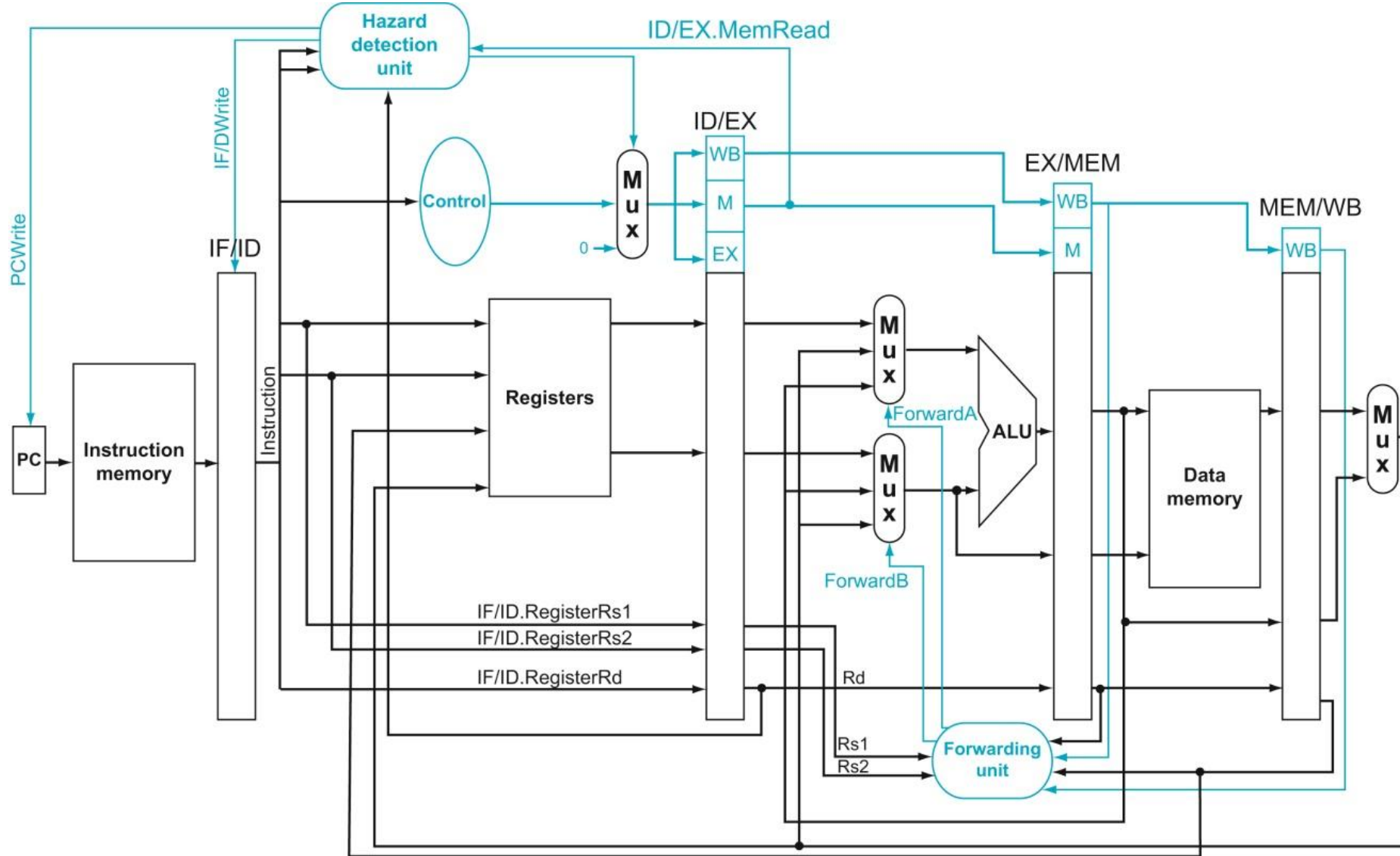
```

if (ID/EX.MemRead == 1 &&
    ((ID/EX.Register_rd == IF/ID.Register_rs1) ||
     (ID/EX.Register_rd == IF/ID.Register_rs2))
    && ID/EX.Register_rd != 0)

```

then stall

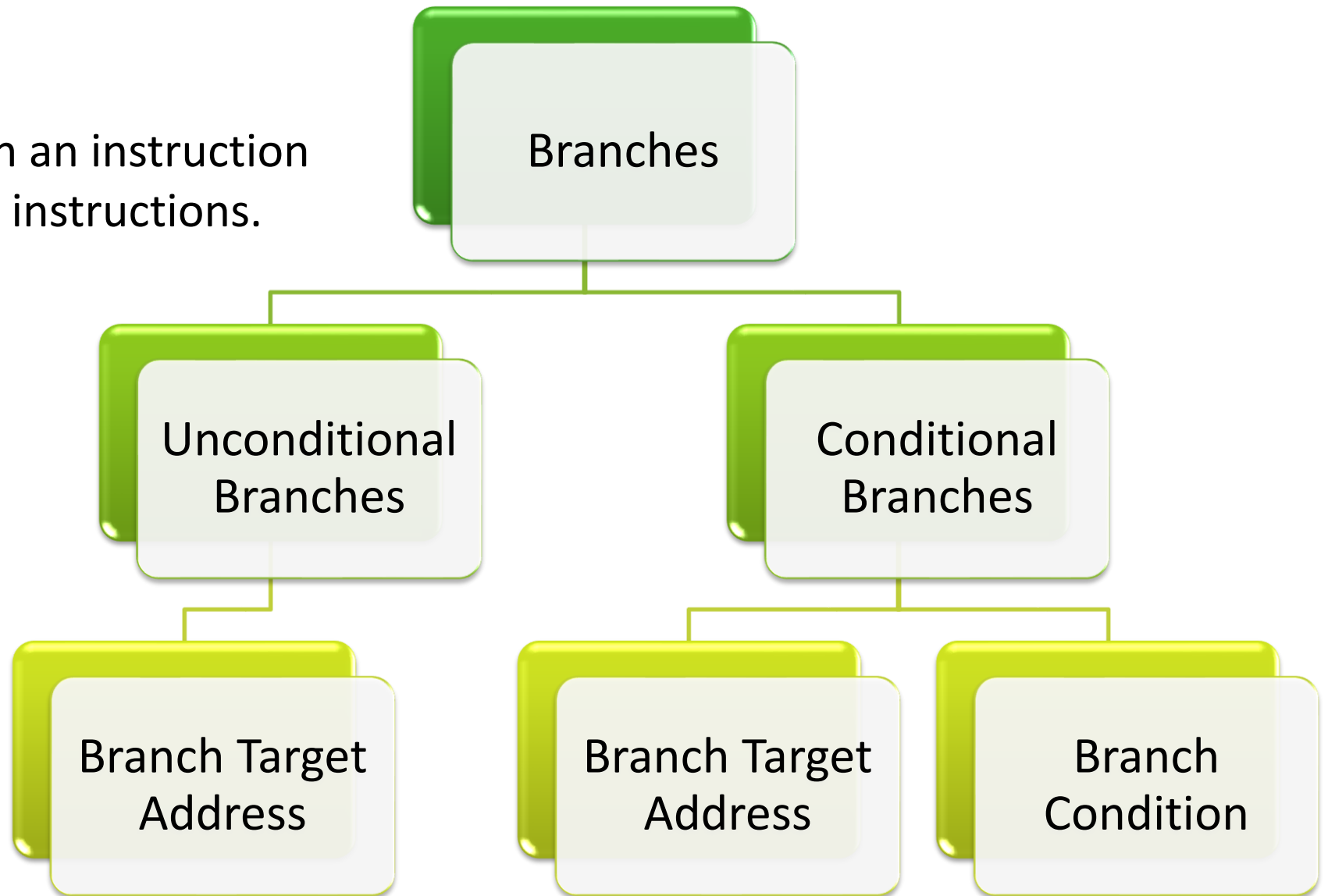
Modified Datapath with HDU



Control Hazards

- Control hazards arise in an instruction pipeline due to branch instructions.

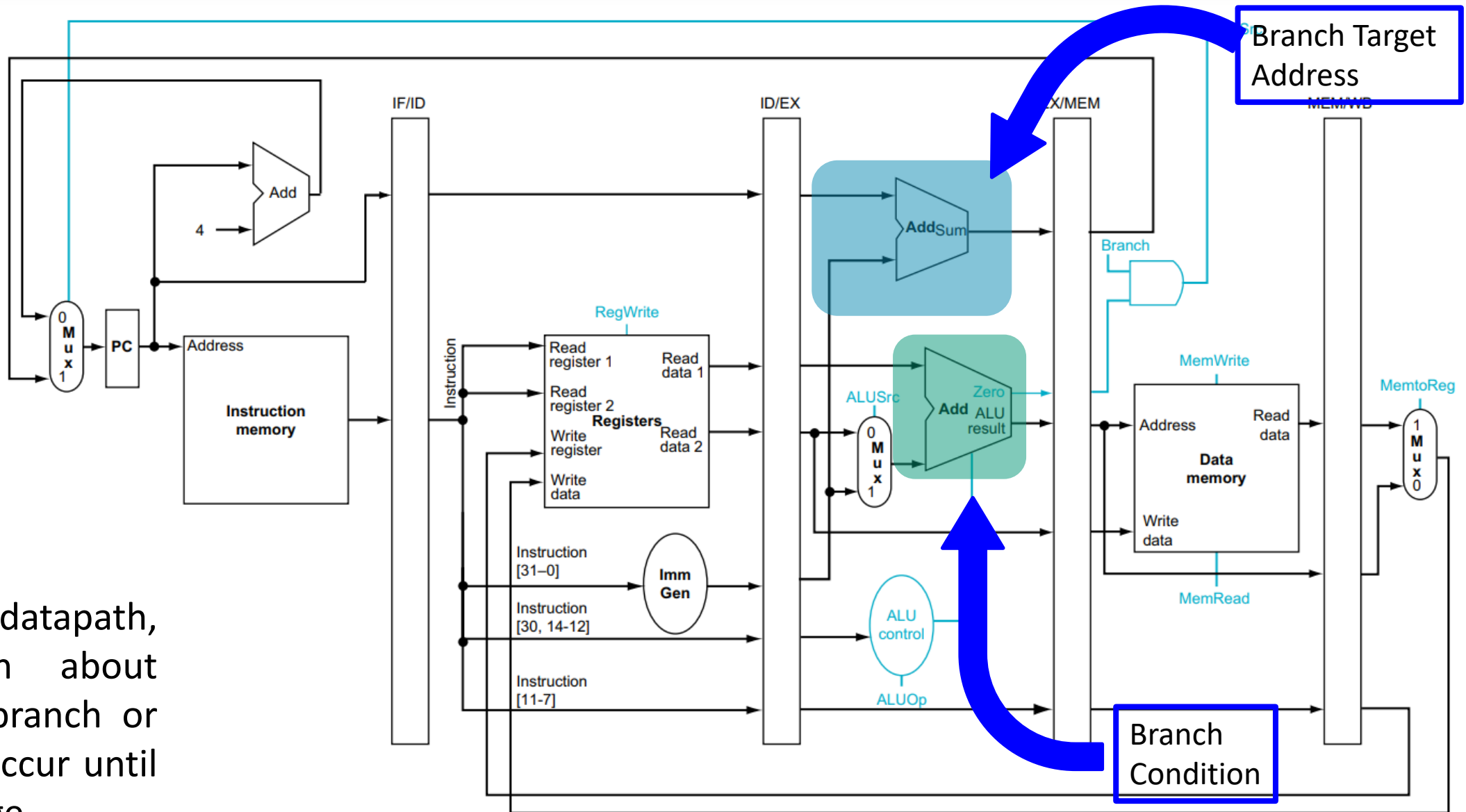
Types of Branches



Branch Operations

Control Hazards

- In the RISC-V datapath, the decision about whether to branch or not doesn't occur until the MEM stage.



Control Hazards

- Consider this sequence:

36: sub x10, x4, x8

40: beq x1, x3, 16

44: and x12, x2, x5

48: or x13, x2, x6

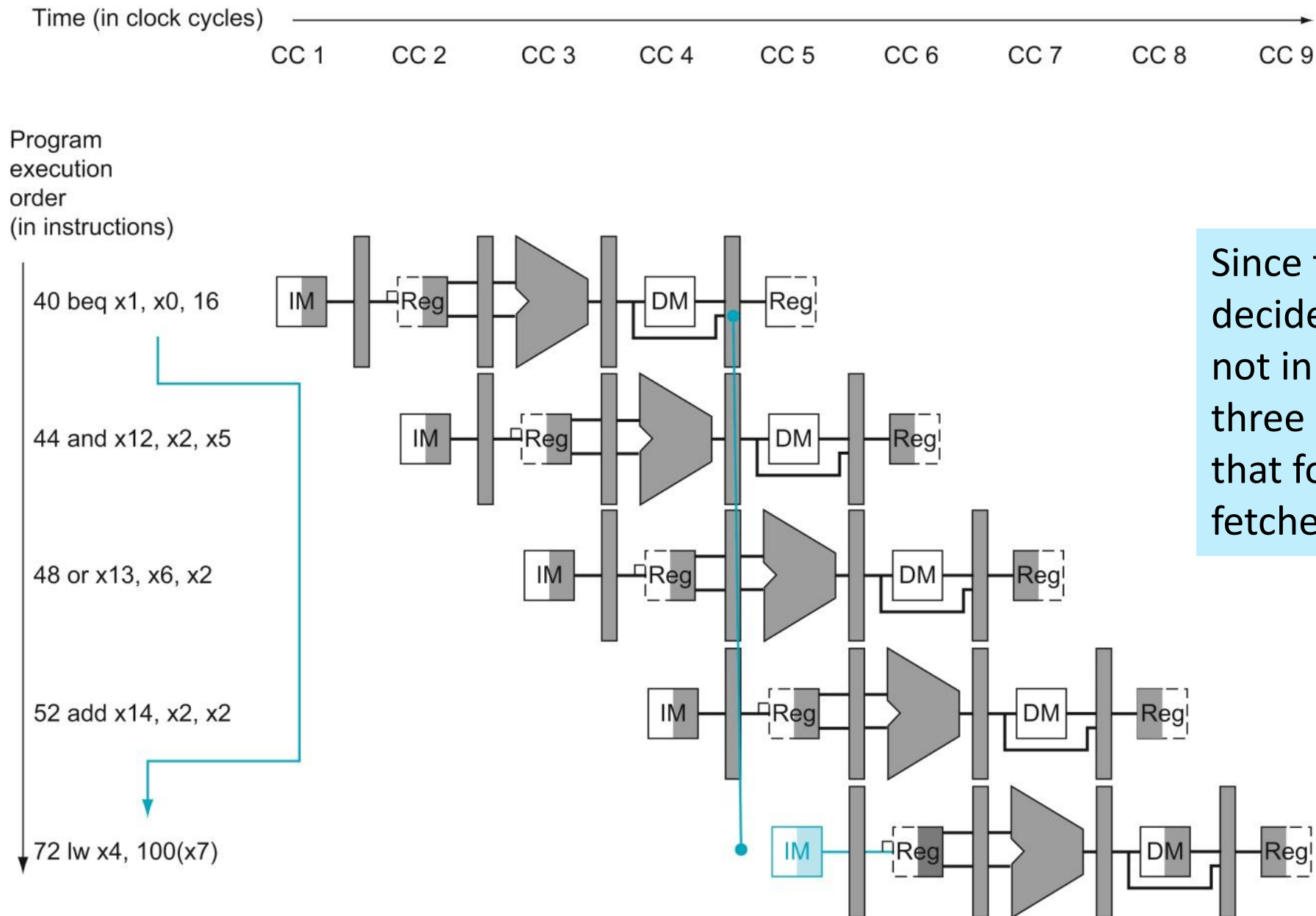
52: add x14, x4, x2

56: sub x15, x6, x7

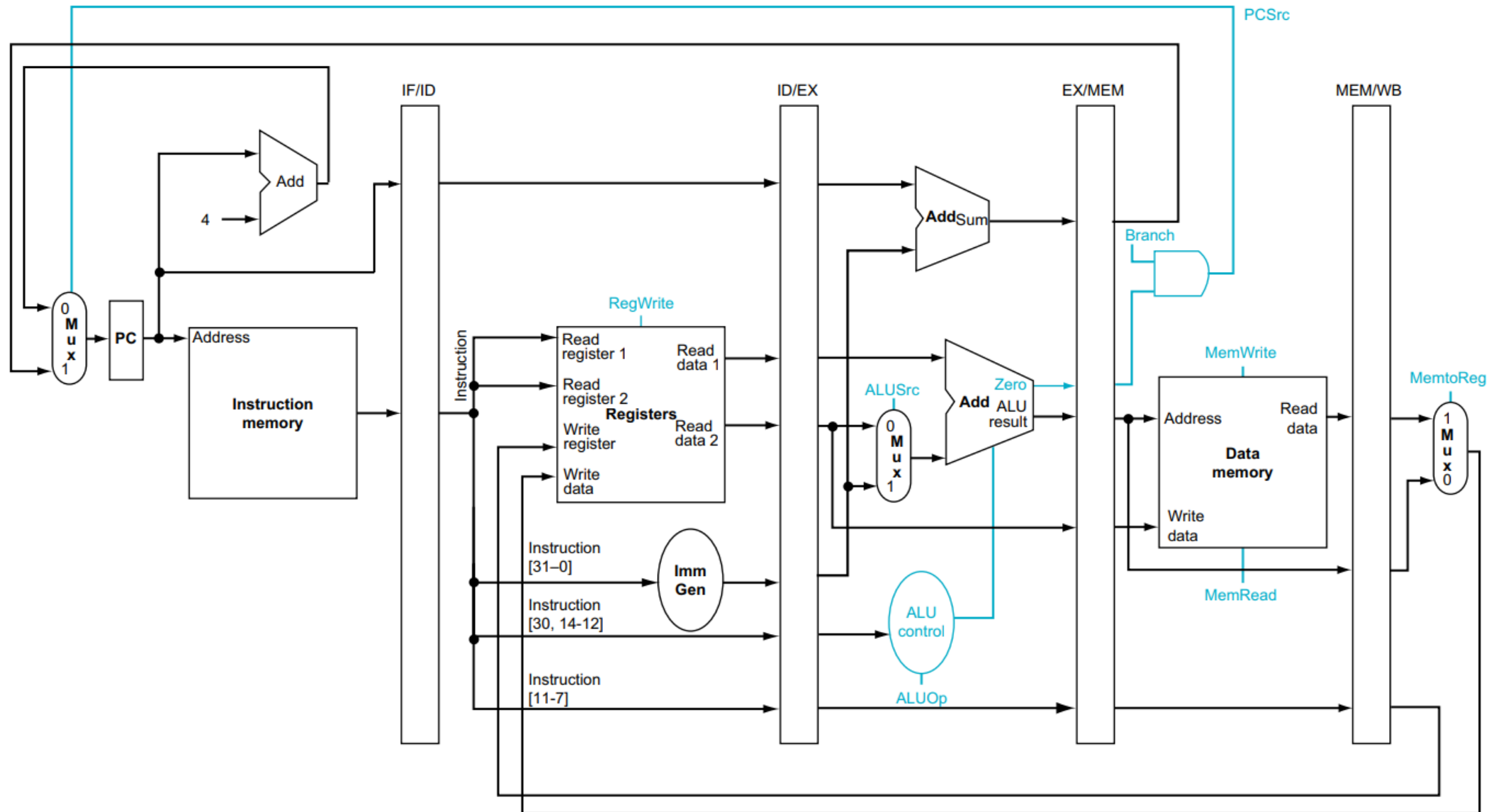
...

72: ld x4, 50(x7)

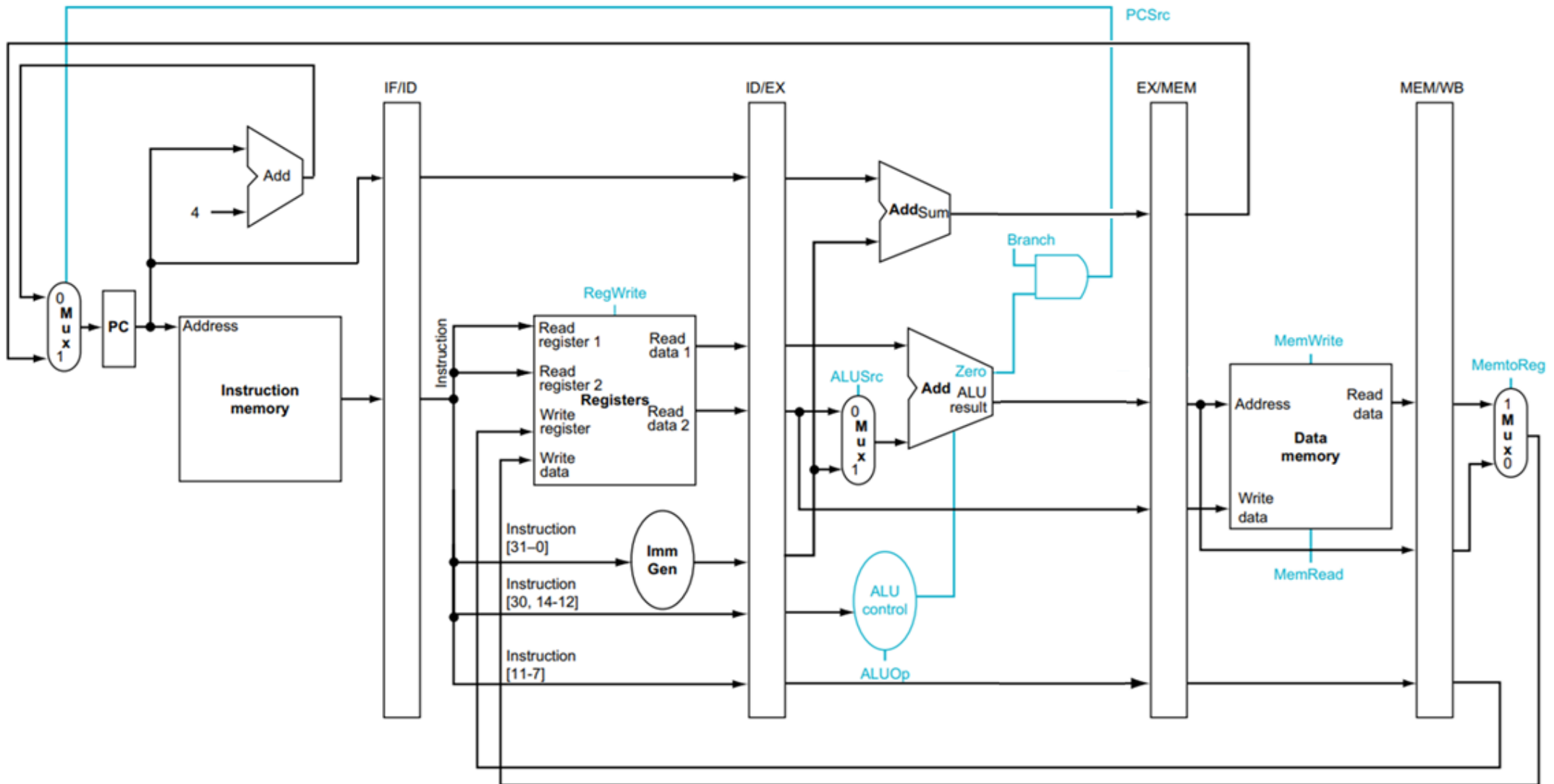
// PC-relative branch to $40 + 16 * 2 = 72$



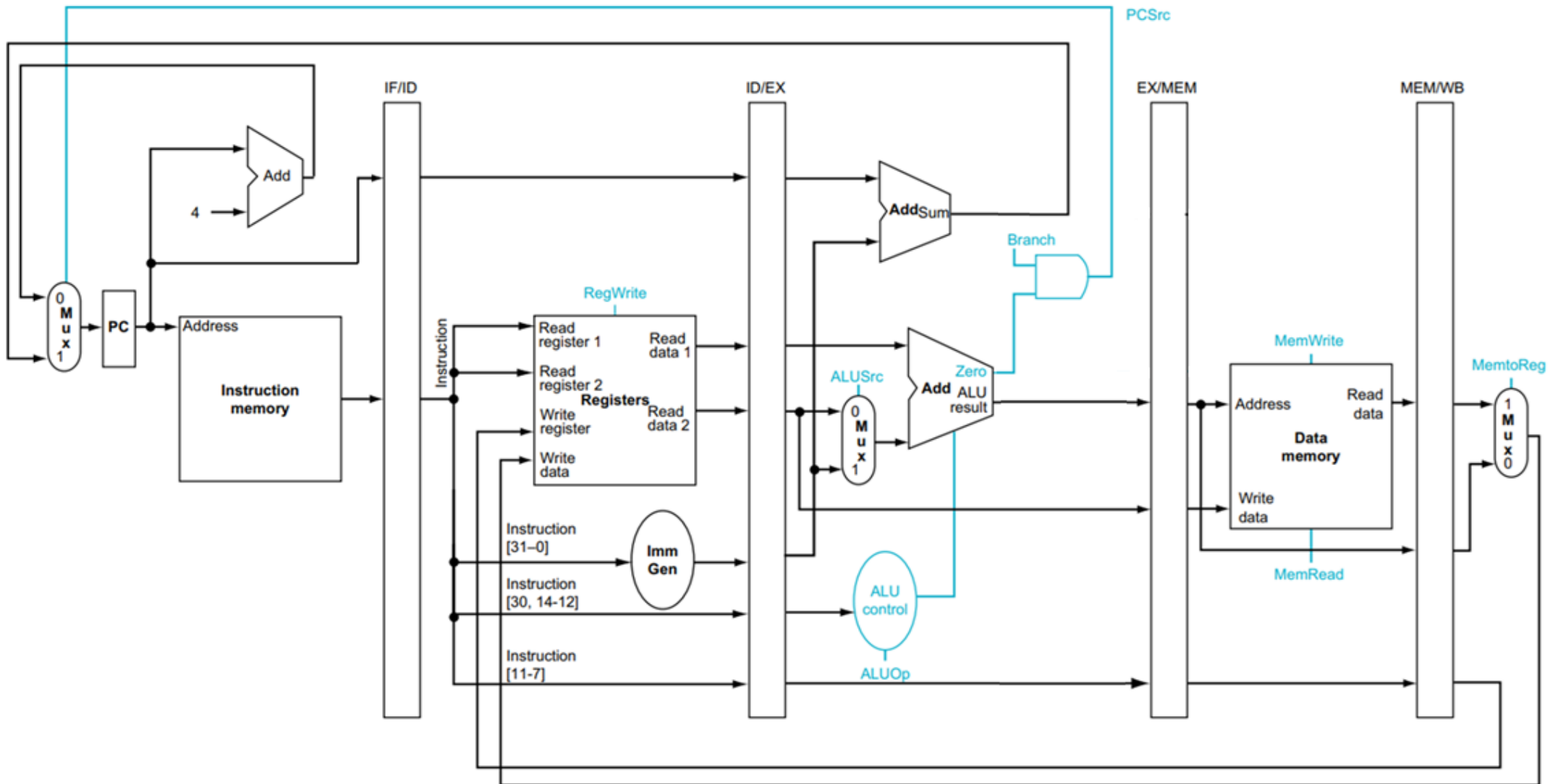
Reducing Branch Delays



Reducing Branch Delays



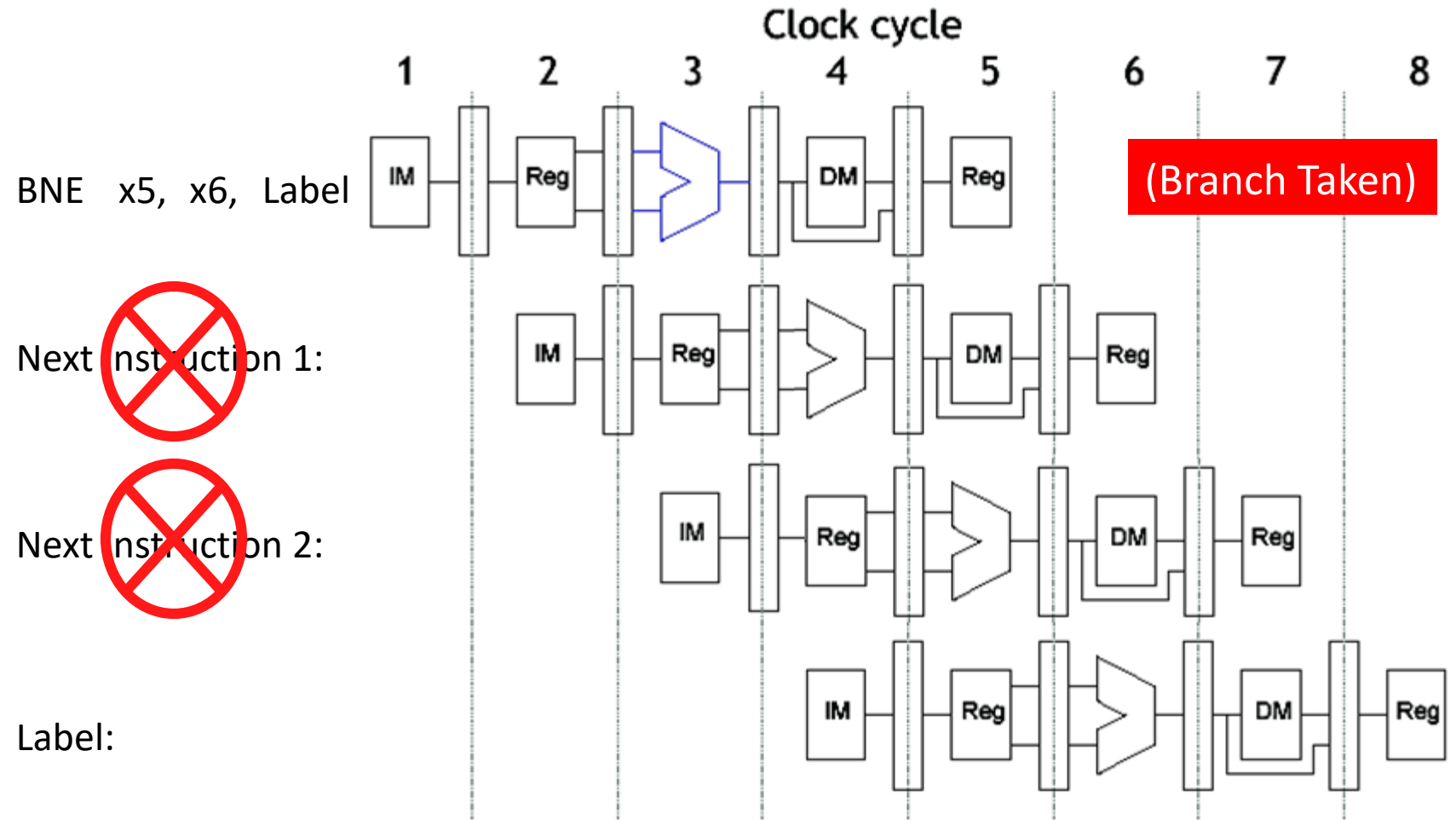
Reducing Branch Delays



Control Hazards

- Until we know the outcome of branch's condition, two following instructions have already made their way into the pipeline.

- Control hazards can cause greater performance loss than data hazards



Reducing Branch Delays

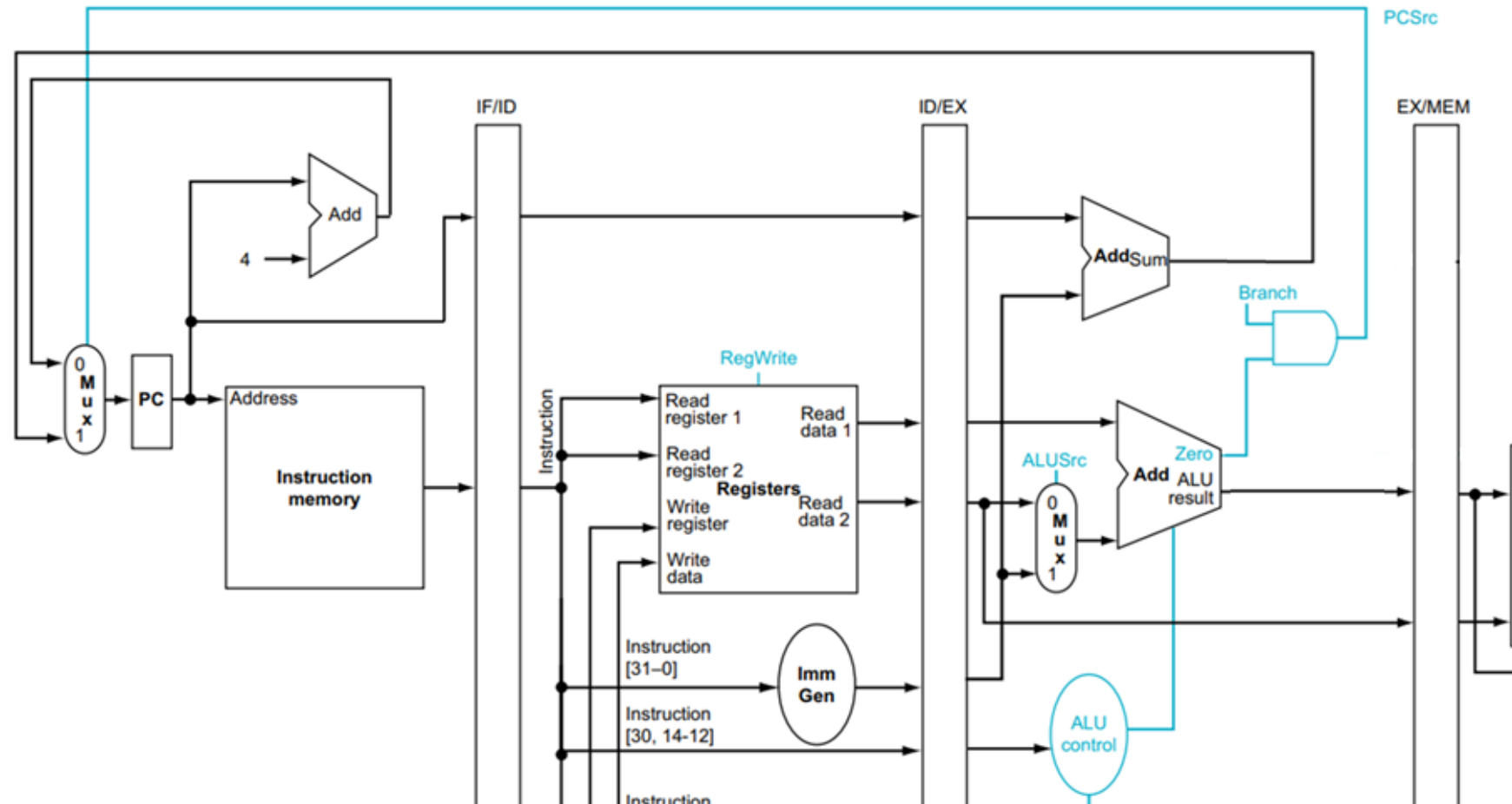
- To improve branch performance the cost of taken branch should be reduced
- If somehow we move the branch related operations earlier in the pipeline, then fewer instructions need to be removed in case of a taken branch.
- Moving the branch decision up requires two actions to occur earlier
 - a) Computing the branch target address
 - b) Evaluating the branch decision

Reducing Branch Delays

a) Computing the branch target address

- Computing the branch target address is easier to implement than evaluating branch decisions

- PC and immediate field is available in the IF/ID pipeline register
- Adder can be moved from the EX stage to the ID stage
- Branch target address calculation will be performed for all instructions, but will only be used when necessary

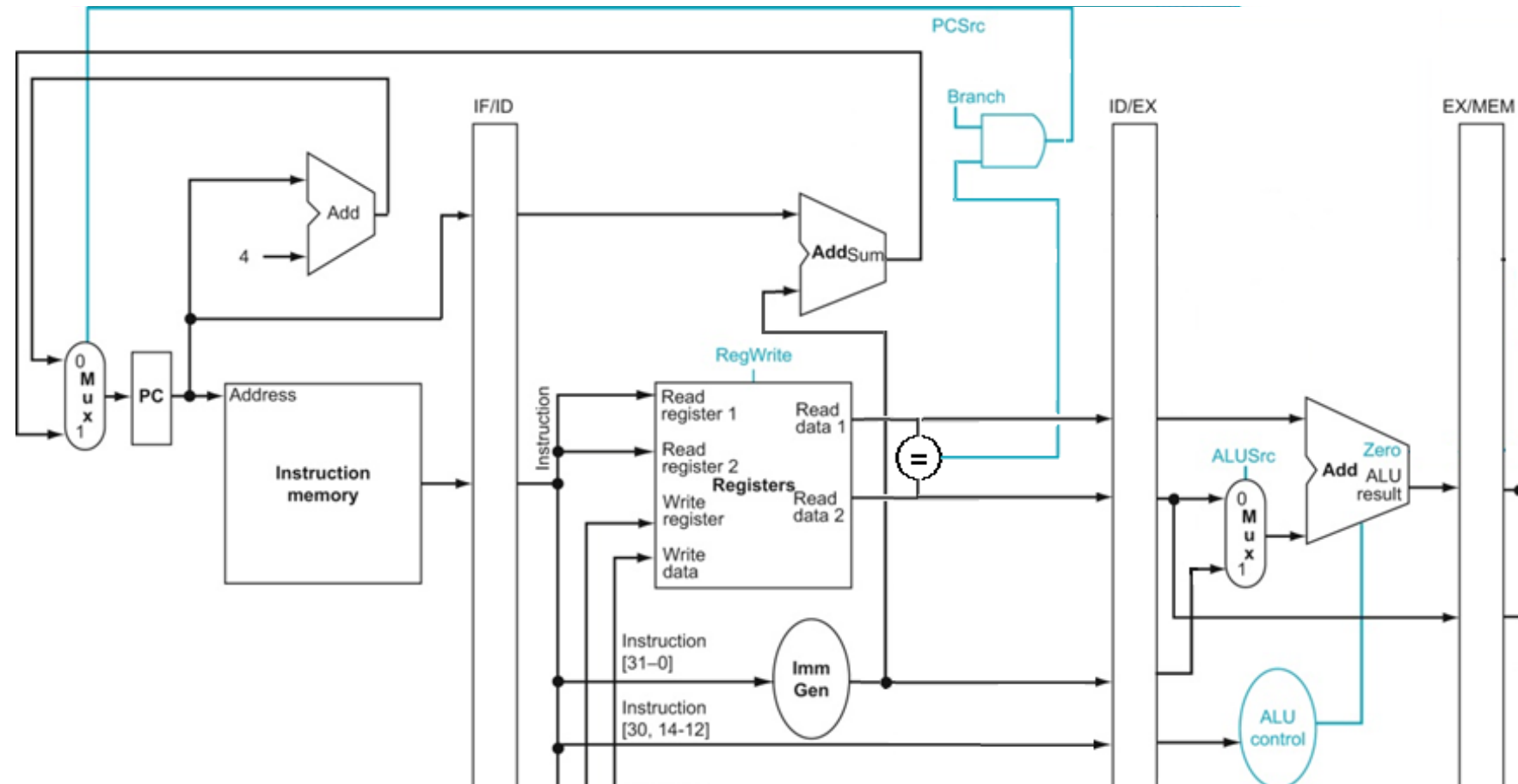


Reducing Branch Delays

b) Evaluating the branch decision

- Branches in RISC-V usually rely on simple tests (equality or sign) and such tests do not require full ALU operation and so can be done using just few gates

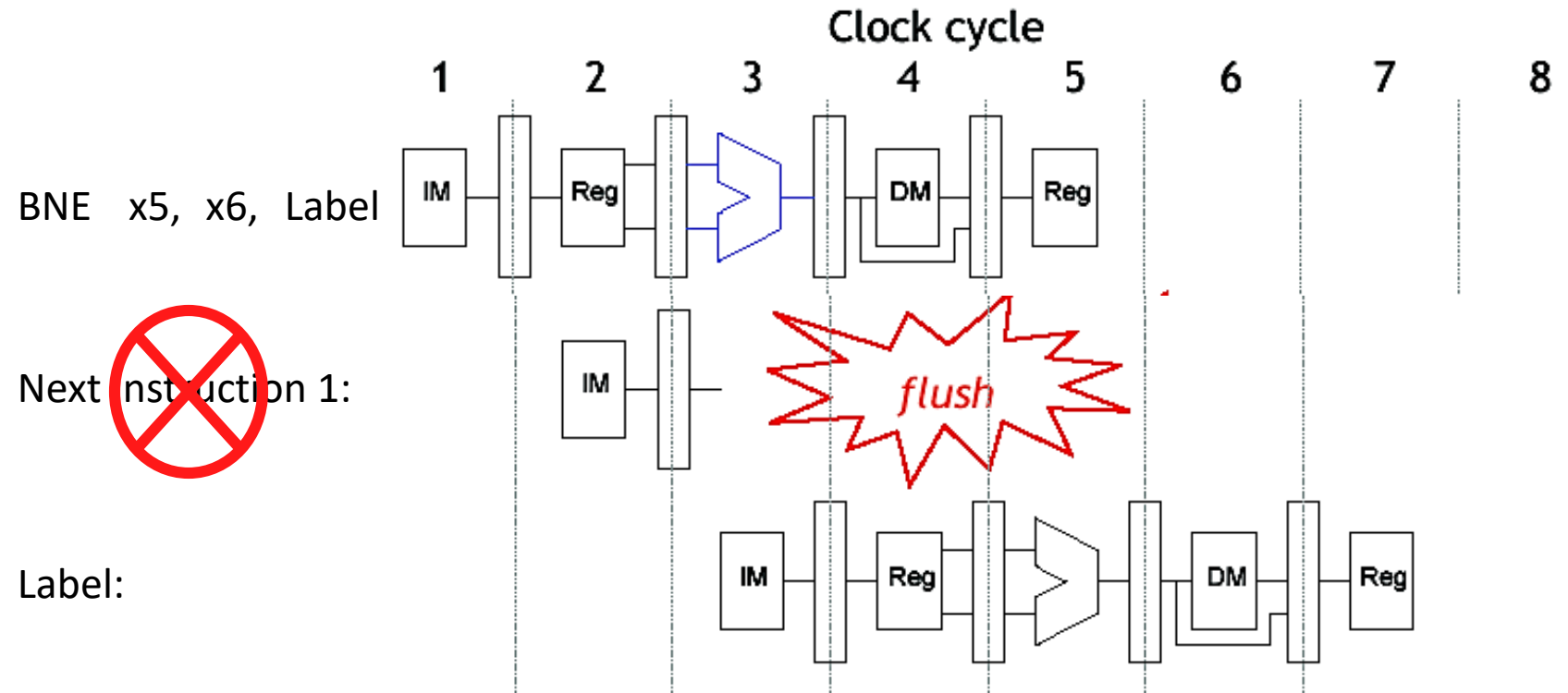
- For branch equal, less than or greater than checks the two registers could be compared at the end of the ID stage once the registers have been read.
- We are assuming that this comparison operation at the end of cycle does not increase the cycle time



Reducing Branch Delays

- With these changes only one instruction will be in the pipeline.

- If the Branch is taken we need to flush this instruction while it's in the IF stage.



Control Hazards

- Consider this sequence:

36: sub x10, x4, x8

40: beq x1, x3, 16

44: and x12, x2, x5

48: or x13, x2, x6

52: add x14, x4, x2

56: sub x15, x6, x7

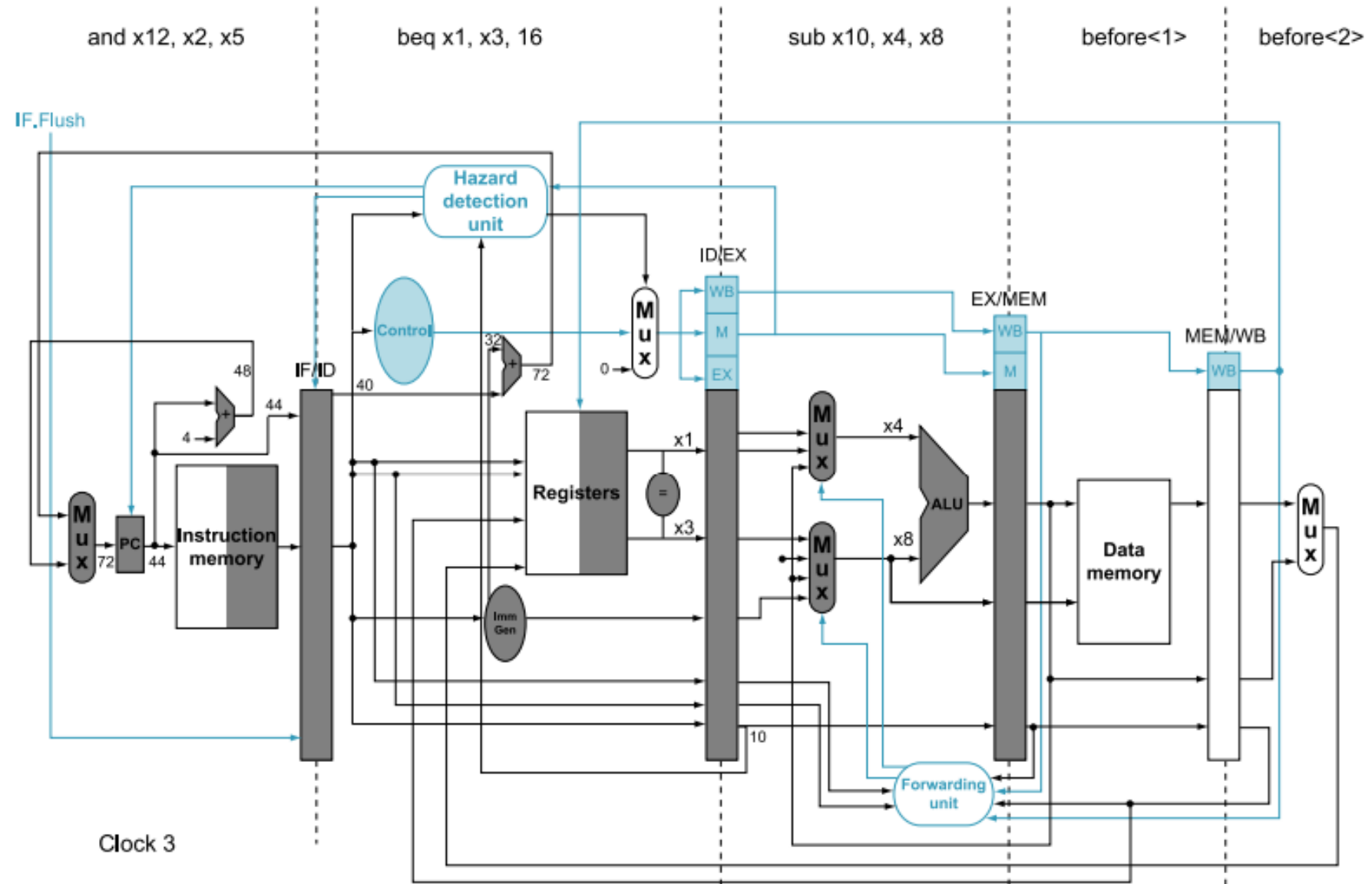
...

72: ld x4, 50(x7)

// PC-relative branch to $40 + 16 * 2 = 72$

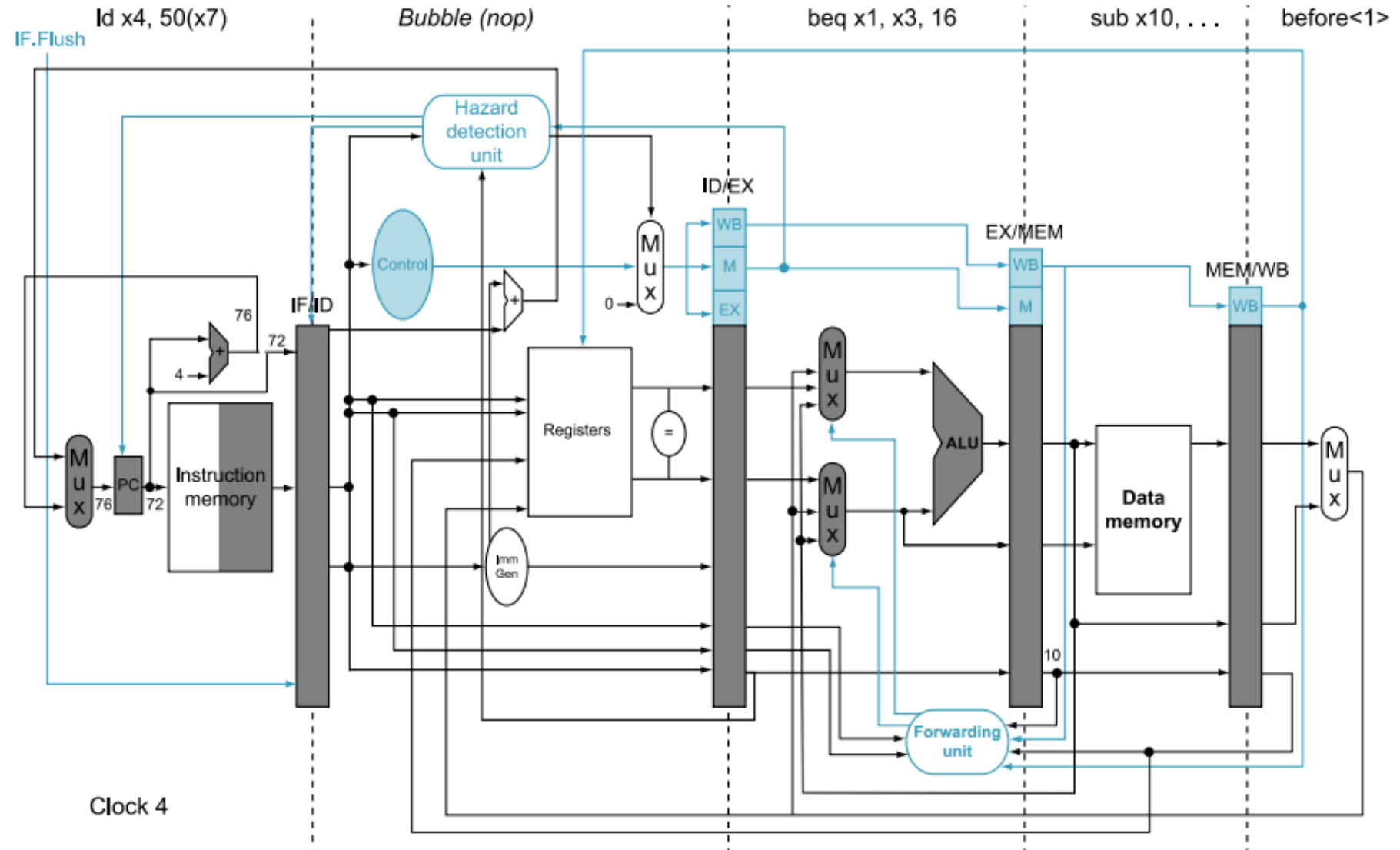
Reducing Branch Delays

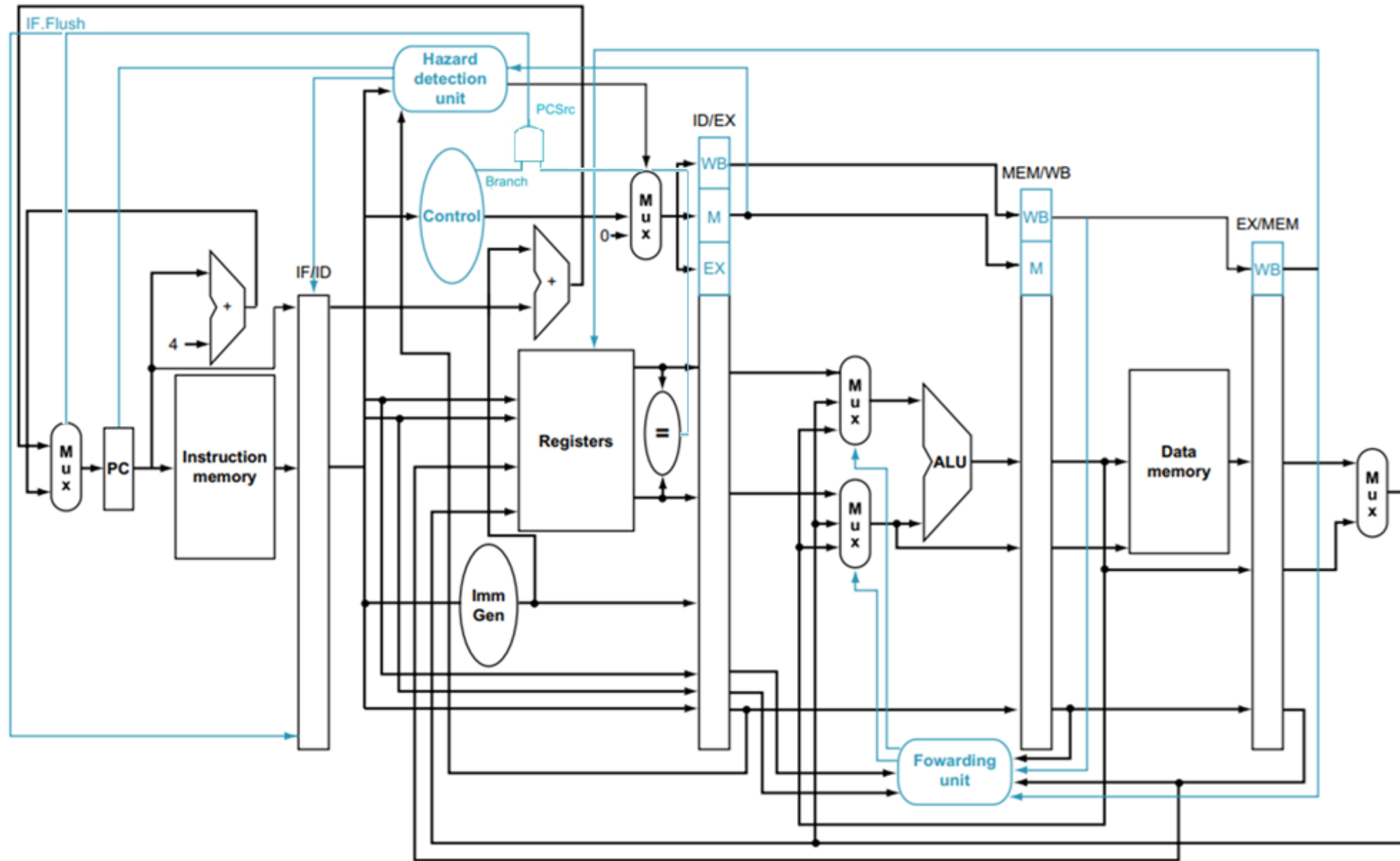
- To flush instructions in the IF stage, we add a control line, called IF.Flush, that clears the IF/ID pipeline register.



Reducing Branch Delays

- To flush instructions in the IF stage, we add a control line, called IF.Flush, that clears the IF/ID pipeline register.
- “Clearing the register” here means we are transforming it into a nop instruction.





Final 5-stage Pipeline

Including

- Forwarding Unit
- Hazard Detection Unit
- IF.Flush signal

Pipeline Hazards

- No Structural Hazards
- One stall for Load-Use Hazard
- One stall for Control Hazards